Introduction to Triggers

A trigger is a special type of stored program that automatically executes (fires) in response to a specific event — INSERT, UPDATE, or DELETE — occurring on a specified table. Unlike a stored procedure, a trigger is never called directly; the database engine invokes it automatically when the triggering event happens.

General Syntax

```
DELIMITER //

CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- trigger logic
    -- use NEW.column_name for inserted/updated values
    -- use OLD.column_name for deleted/previous values
END //

DELIMITER ;
```

Keyword	Meaning
BEFORE	Trigger fires before the triggering event is applied to the table. Often used for validation.
AFTER	Trigger fires after the triggering event has been applied. Often used for auditing/logging.
NEW	Refers to the new row values. Available in INSERT and UPDATE triggers.
OLD	Refers to the row values before the change. Available in UPDATE and DELETE triggers.

FOR EACH ROW	Required in MySQL — the trigger body runs once per affected row.

Database Schema

The examples in this document use the following Employees table and an Employee_Audit table used to log insert, update, and delete activity.

Employees Table:

```
CREATE TABLE Employees (
        emp_id INT PRIMARY KEY,
        emp_name VARCHAR(50),
        department VARCHAR(30),
        salary DECIMAL(10,2),
        experience INT,
        city VARCHAR(30)
);


INSERT INTO Employees VALUES
(101,'Rahul','IT',65000,5,'Hyderabad'),
(102,'Priya','HR',45000,3,'Chennai'),
(103,'Amit','Finance',70000,6,'Bangalore'),
(104,'Sneha','IT',80000,8,'Hyderabad'),
(105,'Kiran','Sales',40000,2,'Pune');
```

Employee_Audit Table:

```
CREATE TABLE Employee_Audit(
        audit_id INT AUTO_INCREMENT PRIMARY KEY,
        emp_id INT,
        action_type VARCHAR(20),
        old_salary DECIMAL(10,2),
        new_salary DECIMAL(10,2),
```

```
    action_time DATETIME
);
```

Trigger Examples

1. BEFORE INSERT (Validation)

Question:

Do not allow employees with salary less than ₹20,000.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_before_insert_salary
BEFORE INSERT
ON Employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 20000 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT='Salary cannot be less than 20000';
    END IF;
END//

DELIMITER ;
```

Test:

```
INSERT INTO Employees
VALUES(106,'Arjun','IT',15000,2,'Delhi');
```

Output:

```
ERROR 1644 (45000):
```

Salary cannot be less than 20000

2. BEFORE INSERT

Question:

Convert employee name into uppercase before inserting.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_before_insert_uppercase
BEFORE INSERT
ON Employees
FOR EACH ROW
BEGIN
        SET NEW.emp_name=UPPER(NEW.emp_name);
END//

DELIMITER ;
```

Test:

```
INSERT INTO Employees
VALUES(106,'arjun','IT',50000,2,'Delhi');
```

Output:

ARJUN

3. AFTER INSERT (Auditing)

Question:

Store inserted employee details in the audit table.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_after_insert
```

```
AFTER INSERT
ON Employees
FOR EACH ROW
BEGIN
        INSERT INTO Employee_Audit
        (emp_id,action_type,new_salary,action_time)
        VALUES
        (NEW.emp_id,'INSERT',NEW.salary,NOW());
END//

DELIMITER ;
```

Test:

```
INSERT INTO Employees
VALUES(107,'Kavya','HR',55000,4,'Chennai');
SELECT * FROM Employee_Audit;
```

4. BEFORE UPDATE (Validation)

Question:

Do not allow salary reduction.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_before_update_salary
BEFORE UPDATE
ON Employees
FOR EACH ROW
BEGIN
        IF NEW.salary<OLD.salary THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT='Salary cannot be reduced';
        END IF;
END//

DELIMITER ;
```

Test:

```
UPDATE Employees
SET salary=30000
WHERE emp_id=101;
```

Output:

Salary cannot be reduced

5. AFTER UPDATE (Audit Salary Change)

Question:

Store old salary and new salary after updating.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_after_update
AFTER UPDATE
ON Employees
FOR EACH ROW
BEGIN
        INSERT INTO Employee_Audit
   (emp_id,action_type,old_salary,new_salary,action_time)
        VALUES
   (OLD.emp_id,'UPDATE',OLD.salary,NEW.salary,NOW());
END//
```

```
DELIMITER ;
```

Test:

```
UPDATE Employees
SET salary=70000
WHERE emp_id=101;
```

6. BEFORE DELETE (Validation)

Question:

Do not allow deleting IT employees.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_before_delete
BEFORE DELETE
ON Employees
FOR EACH ROW
BEGIN
        IF OLD.department='IT' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Cannot delete IT employees';
        END IF;
END//

DELIMITER ;
```

Test:

```
DELETE FROM Employees
WHERE emp_id=101;
```

Output:

Cannot delete IT employees

7. AFTER DELETE (Audit)

Question:

Store deleted employee details.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_after_delete
AFTER DELETE
ON Employees
FOR EACH ROW
BEGIN
        INSERT INTO Employee_Audit
        (emp_id,action_type,old_salary,action_time)
        VALUES
        (OLD.emp_id,'DELETE',OLD.salary,NOW());
END//

DELIMITER ;
```

Test:

```
DELETE FROM Employees
WHERE emp_id=105;
SELECT * FROM Employee_Audit;
```

8. BEFORE UPDATE

Question:

Automatically convert city names into uppercase.

Trigger SQL:

```
DELIMITER //
```

```
CREATE TRIGGER trg_city_uppercase
BEFORE UPDATE
ON Employees
FOR EACH ROW
BEGIN
        SET NEW.city=UPPER(NEW.city);
END//

DELIMITER ;
```

9. BEFORE INSERT

Question:

Experience should not be negative.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_check_experience
BEFORE INSERT
ON Employees
FOR EACH ROW
BEGIN
        IF NEW.experience<0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Experience cannot be negative';
        END IF;
END//

DELIMITER ;
```

10. AFTER INSERT

Question:

Maintain employee count.

Create table:

```
CREATE TABLE Employee_Count(
        total_employees INT
);

INSERT INTO Employee_Count VALUES(5);
```

Trigger:

```
DELIMITER //

CREATE TRIGGER trg_employee_count
AFTER INSERT
ON Employees
FOR EACH ROW
BEGIN
        UPDATE Employee_Count
        SET total_employees=total_employees+1;
END//

DELIMITER ;
```

11. BEFORE DELETE

Question:

Prevent deletion if employee salary is greater than 70000.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_high_salary_delete
```

```
BEFORE DELETE
ON Employees
FOR EACH ROW
BEGIN
        IF OLD.salary>70000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='High salary employee cannot be deleted';
        END IF;
END//

DELIMITER ;
```

12. AFTER UPDATE

Question:

Store every update in audit.

Trigger SQL:

```
DELIMITER //

CREATE TRIGGER trg_update_log
AFTER UPDATE
ON Employees
FOR EACH ROW
BEGIN
        INSERT INTO Employee_Audit
   (emp_id,action_type,old_salary,new_salary,action_time)
        VALUES
   (NEW.emp_id,'UPDATED',OLD.salary,NEW.salary,NOW());
END//
```

DELIMITER ;

DROP TRIGGER

Drop a specific trigger:

DROP TRIGGER trg_after_insert;

Drop another trigger:

DROP TRIGGER trg_before_update_salary;

Check existing triggers:

SHOW TRIGGERS;

Show trigger definition:

SHOW CREATE TRIGGER trg_after_insert;

Real-Time Trigger Use Cases

Scenario	Trigger Type
Prevent invalid salary	BEFORE INSERT
Convert names to uppercase	BEFORE INSERT
Validate experience	BEFORE INSERT
Prevent salary reduction	BEFORE UPDATE
Log salary changes	AFTER UPDATE
Prevent deleting important employees	BEFORE DELETE
Log deleted records	AFTER DELETE
Maintain audit history	AFTER INSERT/UPDATE/DELETE
Update employee count	AFTER INSERT
Validate business rules	BEFORE INSERT/UPDATE

Interview Questions

1. Create a trigger to prevent inserting employees with salary less than ₹25,000.
2. Create a trigger to convert employee names to uppercase before insertion.
3. Create a trigger to log every inserted employee into an audit table.
4. Create a trigger to prevent reducing an employee's salary.
5. Create a trigger to log old and new salaries after an update.
6. Create a trigger to prevent deleting employees from the IT department.
7. Create a trigger to store deleted employee details in an audit table.
8. Create a trigger to ensure experience is never negative.
9. Create a trigger to maintain the total employee count after each insertion.
10. Create a trigger to prevent deletion of employees whose salary is greater than ₹70,000.

PART 3 — QUICK REFERENCE

10. Stored Procedure vs Trigger — Comparison

Aspect	Stored Procedure	Trigger
Invocation	Explicitly called using CALL	Fires automatically on INSERT/UPDATE/DELETE
Parameters	Accepts IN, OUT, INOUT parameters	No parameters; uses NEW / OLD
Can be called manually?	Yes	No
Return value	Via OUT/INOUT params or result set	None (cannot return values to caller)
Typical use	Reusable business logic, batch operations	Auditing, validation, enforcing rules automatically

Drop statement	DROP PROCEDURE	DROP TRIGGER

11. Command Cheat-Sheet

Task	Statement
Create a procedure	CREATE PROCEDURE name(params) BEGIN ... END;
Call a procedure	CALL name(args);
Delete a procedure	DROP PROCEDURE [IF EXISTS] name;
View procedure code	SHOW CREATE PROCEDURE name;
List all procedures	SHOW PROCEDURE STATUS WHERE Db = 'db_name';
Create a trigger	CREATE TRIGGER name {BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON table FOR EACH ROW BEGIN ... END;
Delete a trigger	DROP TRIGGER [IF EXISTS] name;
List all triggers	SHOW TRIGGERS;

QUESTION & ANSWER BANK

Question	Answer
Q1. What is a stored procedure?	A precompiled, named set of SQL statements stored in the database that can be executed using the CALL statement. It can accept parameters and contain procedural logic such as variables, conditions, and loops.
Q2. What is the default parameter mode if none	IN. If no mode is written before a parameter name, MySQL treats it as an IN parameter.

is specified in CREATE PROCEDURE?	
Q3. What is the difference between IN, OUT, and INOUT parameters?	IN passes a value into the procedure only (read-only inside). OUT is set inside the procedure and returned to the caller (initial value ignored). INOUT passes a value in, allows it to be modified, and returns the updated value to the caller.
Q4. Why do we use DELIMITER // before creating a procedure or trigger?	Because the procedure/trigger body itself contains semicolons to separate statements. Changing the delimiter to // (or similar) tells the client to treat the entire block as one statement, ending only at //, so the internal semicolons don't terminate the CREATE statement early.
Q5. How do you call a stored procedure that has an OUT parameter?	Declare a session variable (e.g., @result), pass it in the CALL, then SELECT the variable afterward: CALL proc(@result); SELECT @result;
Q6. How do you permanently remove a stored procedure?	Use DROP PROCEDURE [IF EXISTS] procedure_name;
Q7. What is a trigger in SQL?	A database object that automatically executes a defined block of SQL in response to an INSERT, UPDATE, or DELETE event on a specified table, without being called explicitly.
Q8. What is the difference between a BEFORE trigger and an AFTER trigger?	A BEFORE trigger fires before the change is applied to the table and can validate/modify the incoming row (commonly used for validation). An AFTER trigger fires once the change has been committed and is commonly used for auditing/logging, since it cannot alter the row anymore.
Q9. What do NEW and OLD refer to in a trigger?	NEW refers to the new/incoming row values (available in INSERT and UPDATE triggers). OLD refers to the row's values before the change (available in UPDATE and DELETE triggers). INSERT triggers have no OLD; DELETE triggers have no NEW.

Q10. Can a trigger call a stored procedure or return a value to the caller?	A trigger can call certain stored procedures/functions internally in some databases, but it can never return a value directly to the statement that fired it (unlike a stored procedure called with CALL).
Q11. Why use FOR EACH ROW in a MySQL trigger?	MySQL only supports row-level triggers, so FOR EACH ROW is mandatory — it specifies that the trigger body executes once for every row affected by the INSERT, UPDATE, or DELETE statement.
Q12. How would you reject an invalid INSERT using a trigger?	Create a BEFORE INSERT trigger that checks the NEW row values with an IF condition and raises a custom error using SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'error message'; this aborts the insert.
Q13. Give a real-world example of using a trigger for auditing.	An AFTER UPDATE trigger on an employees table that inserts the emp_id, old_salary, new_salary, and timestamp into a salary_history table every time a salary changes.
Q14. Give a real-world example of using a trigger for validation.	A BEFORE INSERT trigger on a products table that rejects any row where quantity is negative or price is less than or equal to zero, using SIGNAL to raise an error.
Q15. How do you remove a trigger from a database?	Use DROP TRIGGER [IF EXISTS] trigger_name; This removes only the trigger definition, not the table or its data.
Q16. What is the main advantage of stored procedures over sending raw SQL from an application?	Reduced network traffic (single CALL instead of multiple statements), improved performance through precompilation/caching, centralized and reusable business logic, and finer-grained security via EXECUTE privileges instead of direct table access.
Q17. Can a single table have multiple triggers for the same event (e.g., two AFTER INSERT triggers)?	MySQL allows multiple triggers for the same event and timing from version 5.7.2 onward, executed in the order they were created (or as specified with FOLLOWS/PRECEDES); many other databases also support this with similar ordering rules.

Q18. What happens to existing data in a table when its trigger is dropped?	Nothing — DROP TRIGGER only removes the automatic firing logic. All existing rows and data in the table remain completely untouched.